Multi – level linked list

–Insert At Beginning

–Space Complexity and Algorithm

Program:

```
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}
// --- 1. Insert at Beginning (Main Level) ---
void insertAtBeginning()
{
    int data;
    cout << "Enter data: ";
    cin >> data;

    Node *newNode = createNode(data);
    newNode->next = head;
    head = newNode;
    cout << "Inserted " << data << " at beginning." << endl;
}
……
        cout << "1. Insert at Beginning (Main)" << endl;
       case 1:
            insertAtBeginning();
            break;
```

1. ***Input Space Complexity***

   - ***Parameter – only Definition***:

Considering only function parameters:

```
Node *createNode(int data)
```

Parameter memory:

- ***one integer***
- ***constant size***

Hence:

$$O(1)$$

B) Holistic / Full Data Definition Space Complexity

This includes the entire data structure after operation.

The linked list stores:

- $n+1$ ***nodes*** $(1$ ***addition for node insertion***$)$.
- ***each node has fixed – size fields***

Thus:

$$\textit{Total memory} \propto (n+1)$$

Ignoring constants:

$$O(n)$$

2. Auxiliary Space Complexity

Auxiliary space means:

Extra temporary memory used by the algorithm excluding the actual input data structure.

Inside function:

```
int data;
Node *newNode;
```

Also in createNode(int data) function:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

Here , both the newNode contains dynamically memory addresses, where as , dynamically allocated memory :

```
(Node *)malloc(sizeof(Node));
```

itself is input space. But not:

```
Node *newNode;
```

No:

- ***recursion.***
- ***loops with growing memory.***
- ***dynamic temporary arrays.***
- ***stacks proportional to*** n.

So extra temporary memory remains constant.

Hence:

$$\boxed{O(1)}$$

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{Auxiliary} + \underbrace{O(1)}_{Input} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{Auxiliary} + \underbrace{O(n)}_{Input} = O(n).$$

1) If Asked "Total Space Complexity" then natual answer is : O(n) [*Full Data + Auxiliary*]

2) *If not asked total space complexity then*:

> In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity (O(1))** is the expected answer.

In most academic DSA contexts and coding interviews, when someone simply asks:

“What is the space complexity?”

they usually mean:

Auxiliary Space Complexity

unless they explicitly say:

- total space complexity
- overall memory usage
- data structure space
- input space
- storage complexity

So for your insertion function, the expected answer is usually:

$$O(1)$$

because only constant extra working memory is used.

Algorithm:

MultiINSBEG():

1. *Declare an integer type variable data.*
2. *Read data.*
3. *SET NEWPTR* := CALL *CreateNode(data)*

Translating CreateNode(data)

(((***Node*** ∗)***malloc***(***sizeof***(***Node***)) ***is availability of space for insertion of Data, hence, lets name it AVAIL.***))

CreateNode(data):

1. *Allocate New Node*

   SET NewTempPTR := *AVAIL.*

2. *Memory Allocated* ?:

   If NewTempPTR = NULL, then:
   Write: " Error: Memory allocation failed."
   Return and EXIT.
   [*End of If structure.*]

3. *Copy Data*:

   Set INFO[*NewTempPTR*] := *data*

4. *Set Next section of Allocated Memory*:

   Set Link[*NewTempPTR*] := *NULL*

5. *Set Child section of Allocated Memory*:

$$Set\ childLink[NewTempPTR]\ := NULL$$

6. *Return Stored Address in NewTempPTR*

$$return\ NewTempPTR$$

---

4. *SET PREVIOUS ADDRESS HELD by Head Pointer to NEXT of New Node.*

$$SET\ LINK[NEWPTR] := START$$

5. *SET NEWNODE's address to Head Pointer.*

$$SET\ START := NEWPTR$$

6. *Write : ``Inserted``, data, ``at Beginning``.*

7. ***End Of Algorithm***
